

```
[] game.py
>[] main.py
>[] puzzle.py
[text/markdown] README.md
[text/plain] requirements.txt
go through the README.md and tell me how to make the desired
changes to the code
```



How to make the README's changes

I read the README and your code. The code has one intentional defect, plus one accidental bug that will crash the game (below).

Before you change anything: reproduce the bugs

The README wants "before" evidence. In `puzzle.py`, `random.shuffle` produces any permutation, but only half of all permutations are reachable by legal slides. So roughly 50% of your boards are unsolvable.

Run this in the project folder to prove it:

```
python
```

```
# check_parity.py
from puzzle import Puzzle

def solvable(board, n):
    flat = [x for row in board for x in row]
    tiles = [x for x in flat if x]
    inv = sum(1 for i in range(len(tiles)) for j in range(i + 1, len(tiles)) if tiles[i] >
tiles[j])
    if n % 2 == 1:
        return inv % 2 == 0
    blank_row_from_bottom = n - flat.index(0) // n
    return (inv + blank_row_from_bottom) % 2 == 1

bad = sum(not solvable(Puzzle(4).board, 4) for _ in range(1000))
print(f"unsolvable: {bad}/1000") # ~500 before the fix, 0 after
```

Accidental bug in game.py : if `key not in "wasd"` is a substring test. Pressing Enter (`" "`) or typing `wa` passes the check, and then `move()` raises `KeyError` . Fix it with a tuple:
`key not in ("w", "a", "s", "d")` .

Task 1: guaranteed solvable boards

Start from the solved board and apply random legal blank moves. Every reachable state is solvable by construction. Two details matter: don't immediately undo the previous move (it wastes scramble steps), and re-scramble in the rare case the result is already solved.

Task 4: valid-action feedback

Have `move()` return the tile that slid, or `None` if nothing moved. The game then counts a move and prints success only when a tile is returned. This also gives you the Task 2 guard in one place.

Updated puzzle.py

python

```
import random

SIZES = (3, 4, 5)
DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}

class Puzzle:
    def __init__(self, size=4):
        if size not in SIZES:
            raise ValueError(f"size must be one of {SIZES}")
        self.size = size
        self.board = self.make_board()

    def solved_board(self):
        n = self.size
        tiles = list(range(1, n * n)) + [0]
        return [tiles[r * n:(r + 1) * n] for r in range(n)]

    def make_board(self):
        # Scramble with legal moves only, so the result is always solvable.
        while True:
            self.board = self.solved_board()
            last = None
            for _ in range(self.size * self.size * 20):
                options = [d for d in DIRECTIONS
                           if d != OPPOSITE.get(last) and self._can_slide(d)]
                last = random.choice(options)
                self._slide(last)
```

```

        if not self.solved():
            return self.board

    def blank_pos(self):
        for r in range(self.size):
            for c in range(self.size):
                if self.board[r][c] == 0:
                    return r, c

    def _target(self, direction):
        r, c = self.blank_pos()
        dr, dc = DIRECTIONS[direction]
        return r, c, r + dr, c + dc

    def _can_slide(self, direction):
        _, _, nr, nc = self._target(direction)
        return 0 <= nr < self.size and 0 <= nc < self.size

    def _slide(self, direction):
        if not self._can_slide(direction):
            return None
        r, c, nr, nc = self._target(direction)
        tile = self.board[nr][nc]
        self.board[r][c], self.board[nr][nc] = tile, 0
        return tile

    def move(self, direction):
        """Return the tile that slid into the blank, or None if nothing moved."""
        if self.solved() or direction not in DIRECTIONS:  # Task 2 guard
            return None
        return self._slide(direction)

    def solved(self):
        return self.board == self.solved_board()

```

Task 2 and Task 3: lifecycle and session state

The key idea for Task 3 is to separate two things:

- **Board** (`new_board`): only replaces the `Puzzle` .
- **Session** (`start_session`): moves, start time and finish time.

Only `start_session` resets the counters, so recreating a board can never wipe them by accident. For Task 2, freeze the timer at the moment of solving so it stops ticking.

Updated `game.py`

python

```

import time
from puzzle import Puzzle, SIZES

class SlidingPuzzle:
    def __init__(self):
        self.size = 4
        self.puzzle = None
        self.moves = 0
        self.started = None
        self.finished_at = None

    def new_board(self):          # board only; session values untouched
        self.puzzle = Puzzle(self.size)

    def start_session(self, size): # the only place counters are reset
        self.size = size
        self.new_board()
        self.moves = 0
        self.started = time.monotonic()
        self.finished_at = None

    def elapsed(self):
        end = self.finished_at if self.finished_at is not None else time.monotonic()
        return int(end - self.started)

    def choose_size(self):
        while True:
            raw = input(f"Board size {SIZES}: ").strip()
            if raw.isdigit() and int(raw) in SIZES:
                return int(raw)
            print("Please enter 3, 4 or 5.")

    def display(self):
        width = len(str(self.size * self.size - 1))
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or '':>{width}}" for x in row))
        print("Moves:", self.moves, " Time:", self.elapsed(), "s")

    def run(self):
        print("Sliding Puzzle - W/A/S/D moves the tile into the blank. Q quits.")
        self.start_session(self.choose_size())
        while True:
            self.display()
            if self.puzzle.solved():
                print(f"Solved in {self.moves} moves!")
                return

```

```

key = input("> ").strip().lower()
if key == "q":
    return
if key not in ("w", "a", "s", "d"):
    print("Use W/A/S/D (or Q to quit).")
    continue
tile = self.puzzle.move(key)
if tile is None:
    print("That move is not possible.")
else:
    self.moves += 1
    print(f"Slid tile {tile}.")
    if self.puzzle.solved():
        self.finished_at = time.monotonic()

```

`main.py` and `requirements.txt` need no changes.

What to test (the README's "Required testing")

Test	How
Many fresh boards	Rerun <code>check_parity.py</code> for sizes 3, 4 and 5. Expect 0 unsolvable and no board that starts solved.
All sizes	Start the game and pick 3, 4 and 5. Check alignment, since 5x5 has two-digit tiles.
Solve a small board	Temporarily scramble with a few moves (e.g. change <code>* 20</code> to <code>2</code>) and solve it. You should see "Solved", a frozen timer, and the game exiting.
Impossible moves	Push into a wall. Expect "That move is not possible.", with no count increase and no "Slid tile" message.
Moves	Make 3 valid and 2 invalid moves. Moves should read 3.
Timer	It should increase between prompts and stop once solved.
Invalid commands	Try empty Enter, <code>x</code> , <code>wa</code> and <code>12</code> . None should crash, and none should change the count.
Quit	<code>q</code> exits cleanly at any point.

Submission reminders

- **Before video:** show unsolvable boards or the Enter-key crash (for example, run the game several times or press Enter).
- **After video:** show the size menu, a valid move with feedback, an invalid move with no count change, and a solved board.

- Keep the full chat history link. The README also warns against unexplained code, so make sure you understand the parity idea and the return- None design before submitting.

If you'd like, I can write a proper `test_puzzle.py` (using `unittest`, so no dependencies) covering the table above.

2026-09-30 08:35:37 You 

take me through each task one by one



Walking through each task

I'll go in README order. Each task has the goal, the reason, the exact change, and how to check it. Do them one at a time and run the game after each, which also makes for a cleaner chat history.

Task 1: Guarantee solvable starting states

The problem: `random.shuffle` can produce any of the $16!$ orderings, but a legal slide can only reach half of them. Every slide swaps the blank with a neighbouring tile, and this preserves a parity property (the inversion count, adjusted by the blank's row on even-width boards). A shuffled board has the wrong parity about 50% of the time, and then no sequence of moves can ever solve it.

The fix: don't shuffle tiles directly. Build the solved board, then scramble it using only legal blank moves. Every state you reach this way is reachable from solved, so it is solvable by construction.

Change in `puzzle.py` :

1. Add a helper that returns the solved layout:

python

```
def solved_board(self):
    n = self.size
    tiles = list(range(1, n * n)) + [0]
    return [tiles[r * n:(r + 1) * n] for r in range(n)]
```

2. Replace `make_board` so it starts from that layout and makes many random legal moves:

python

```
def make_board(self):
    while True:
```

```

self.board = self.solved_board()
last = None
for _ in range(self.size * self.size * 20):
    options = [d for d in DIRECTIONS
                if d != OPPOSITE.get(last) and self._can_slide(d)]
    last = random.choice(options)
    self._slide(last)
if not self.solved():
    return self.board

```

Two design choices here:

- `OPPOSITE` stops the scramble from undoing its own last step, which would waste moves.
- The `while True` and `if not self.solved()` check covers the rare case where the scramble ends on the solved board, since a puzzle that starts solved would end instantly.

Test: run `check_parity.py` from my earlier message before and after. Before, about 500 of 1000 boards are unsolvable. After, it should be 0. Repeat for sizes 3, 4 and 5.

Task 4: Valid-action feedback (do this before Task 2)

I'm taking this out of order because Tasks 2 and 3 build on it.

The problem: `move()` returns `True` or `False`, but nothing about the game confirms a tile actually moved. The README wants success feedback only when a tile really slid into the blank, and invalid moves must never bump the counter.

The fix: make `move()` return the tile that slid, or `None` if nothing moved. That gives the game something meaningful to print.

In `puzzle.py`, split the logic into a private `_slide` (does the move) and a public `move` (validates first):

python

```

def _slide(self, direction):
    if not self._can_slide(direction):
        return None
    r, c, nr, nc = self._target(direction)
    tile = self.board[nr][nc]
    self.board[r][c], self.board[nr][nc] = tile, 0
    return tile

def move(self, direction):
    if self.solved() or direction not in DIRECTIONS:
        return None
    return self._slide(direction)

```

In `game.py`, use the result:

```
python

tile = self.puzzle.move(key)
if tile is None:
    print("That move is not possible.")
else:
    self.moves += 1
    print(f"Slid tile {tile}.")
```

The `elif`-style structure means the counter and the success message can only ever happen together.

Also fix the input check. `key not in "wasd"` is a substring test, so `""` and `"wa"` pass and then crash `move()` with a `KeyError`. Change it to:

```
python

if key not in ("w", "a", "s", "d"):
```

Test: push into a wall five times, and the count should stay at 0 with no "Slid tile" text. Then make one valid move, and the count should be 1.

Task 2: Complete puzzle lifecycle

What's already there: `solved()` exists and the game checks it at the top of the loop. Two gaps remain:

1. Nothing stops state changes after completion.
2. The timer keeps counting after the puzzle is solved, because `display()` always calls `time.monotonic()`.

Fix 1, the guard. You already added it in Task 4: `move()` starts with `if self.solved(): return None`. Since the guard lives in the puzzle itself, no caller can change a finished board.

Fix 2, freezing the timer. Add `finished_at` and an `elapsed()` method in `game.py`:

```
python

def elapsed(self):
    end = self.finished_at if self.finished_at is not None else time.monotonic()
    return int(end - self.started)
```

Record the finish time right after a successful move solves the board:

```
python
```

```
if self.puzzle.solved():
    self.finished_at = time.monotonic()
```

`display()` then uses `self.elapsed()` instead of computing time inline.

Clean ending: the loop already prints "Solved" and returns. I changed the message to `f"Solved in {self.moves} moves!"` so the final result is visible.

Test: to solve one by hand quickly, temporarily change the scramble count from `* 20` to `2` in `make_board`, play it out, and confirm the timer freezes and the game exits. Change it back afterwards.

Task 3: Size, timer, and move modes

What's asked: 3x3, 4x4 and 5x5 modes, with moves and time tracked consistently, and no accidental resets when only the board is recreated.

The key idea: there are two kinds of state, and they must not be mixed.

- **Board state** is the tile layout.
- **Session state** is `moves`, `started` and `finished_at`.

Right now `__init__` sets both together, so anything that rebuilds the board would be tempted to reset the counters too. Separate them into two methods in `game.py`:

python

```
def new_board(self):                # board only
    self.puzzle = Puzzle(self.size)

def start_session(self, size):      # the only place counters reset
    self.size = size
    self.new_board()
    self.moves = 0
    self.started = time.monotonic()
    self.finished_at = None
```

Size selection. Add a prompt with validation:

python

```
def choose_size(self):
    while True:
        raw = input(f"Board size {SIZES}: ").strip()
        if raw.isdigit() and int(raw) in SIZES:
            return int(raw)
        print("Please enter 3, 4 or 5.")
```

and call `self.start_session(self.choose_size())` at the start of `run()` .

Put `SIZES = (3, 4, 5)` at the top of `puzzle.py` , and have `Puzzle.__init__` raise `ValueError` for anything else, so an invalid size can't slip in from another route.

Display alignment. A 5x5 board has tiles up to 24, so the hard-coded width of 2 works, but computing it is safer:

```
python

width = len(str(self.size * self.size - 1))
print(" ".join(f"{x or '':>{width}}" for x in row))
```

Test: start each size in turn, check the alignment, make a few moves, and confirm the move count and timer behave the same across all three.

Suggested order to work in

1. Reproduce the bug and record the "before" video.
2. Task 1, then run the parity check.
3. Task 4 (the `move()` change and the `wasd` fix).
4. Task 2 (timer freeze and lifecycle).
5. Task 3 (session split and size menu).
6. Run through the README's testing list and record the "after" video.

Want me to go deeper on any one of these, for example a plain-language explanation of the parity proof so you can defend it, or a `unittest` file covering these checks?

2026-09-30 08:44:33 You 

"OPPOSITE" is not defined
Im getting the same message for "DIRECTIONS"



That's my mistake. In the task-by-task walkthrough I used `DIRECTIONS` and `OPPOSITE` without telling you to define them, and I skipped the two small helpers `_target` and `_can_slide` as well. You'll probably hit errors for those next.

Fix 1: add the constants at the top of `puzzle.py`

Put these right under `import random` , above the class:

python

```
import random

SIZES = (3, 4, 5)
DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}
```

- `DIRECTIONS` maps each key to a (row, column) offset, replacing the dictionary that was previously written inline inside `move()` .
- `OPPOSITE` maps each direction to its reverse, so the scramble can avoid undoing its last step.

Fix 2: add the two helpers inside the `Puzzle` class

`make_board` and `_slide` call these, so they need to exist:

python

```
def _target(self, direction):
    r, c = self.blank_pos()
    dr, dc = DIRECTIONS[direction]
    return r, c, r + dr, c + dc

def _can_slide(self, direction):
    _, _, nr, nc = self._target(direction)
    return 0 <= nr < self.size and 0 <= nc < self.size
```

- `_target` returns the blank's current position and the position it would move to.
- `_can_slide` checks that the destination is inside the board.

Check that you have every method

Your `Puzzle` class should now contain all of these:

`__init__` , `solved_board` , `make_board` , `blank_pos` , `_target` , `_can_slide` , `_slide` , `move` , `solved`

If any is missing, you'll get a `NameError` or `AttributeError` for that name. Also make sure `solved()` uses the new version:

python

```
def solved(self):
    return self.board == self.solved_board()
```

Quick test

```
bash
```

```
python -c "from puzzle import Puzzle; p = Puzzle(4); print(p.board)"
```

If that prints a board with no error, `puzzle.py` is working and you can move on to the `game.py` changes. If another error appears, paste the full traceback here and I'll sort it out.

2026-09-30 08:48:03 You 

i have made the suggested changes, but the `check_parity.py` is not running properly now. Also, do i have to make any new changes for the other tasks



Yes, you need more changes: Tasks 2, 3 and 4 are mostly in `game.py`, which you haven't changed yet (if you have, tell me).

`check_parity.py`

I can't see the error, so here are the likely causes, then a better version of the script.

1. **Wrong folder.** `check_parity.py` must be in the same folder as `puzzle.py`, and you must run it from there (`python check_parity.py`).
2. **A traceback from `puzzle.py`** . If a method is missing (see the list in my last message), the script fails inside `Puzzle(...)` . Run `python -c "from puzzle import Puzzle; print(Puzzle(4).board)"` first to confirm.
3. **The output "unsolvable: 0/1000" is the correct result.** Before the fix it showed ~500. Now 0 means the fix works.

Here is a replacement that tests all three sizes and also checks that no board starts solved:

```
python
```

```
# check_parity.py
from puzzle import Puzzle

def solvable(board, n):
    flat = [x for row in board for x in row]
    tiles = [x for x in flat if x]
    inv = sum(1 for i in range(len(tiles))
              for j in range(i + 1, len(tiles)) if tiles[i] > tiles[j])
    if n % 2 == 1:
```

```

        return inv % 2 == 0
    blank_row_from_bottom = n - flat.index(0) // n
    return (inv + blank_row_from_bottom) % 2 == 1

for n in (3, 4, 5):
    bad = already_solved = 0
    for _ in range(1000):
        p = Puzzle(n)
        if not solvable(p.board, n):
            bad += 1
        if p.solved():
            already_solved += 1
    print(f"{n}x{n}: unsolvable={bad}/1000, started solved={already_solved}/1000")

```

Expected output is 0 and 0 on every line. If you see an error instead, paste the full traceback.

What's left to change: `game.py`

Replace the whole file with this. It covers Tasks 2, 3 and 4 and the `wasd` crash fix:

python

```

import time
from puzzle import Puzzle, SIZES

class SlidingPuzzle:
    def __init__(self):
        self.size = 4
        self.puzzle = None
        self.moves = 0
        self.started = None
        self.finished_at = None

    def new_board(self):          # board only; session values untouched
        self.puzzle = Puzzle(self.size)

    def start_session(self, size): # the only place counters are reset
        self.size = size
        self.new_board()
        self.moves = 0
        self.started = time.monotonic()
        self.finished_at = None

    def elapsed(self):
        end = self.finished_at if self.finished_at is not None else time.monotonic()
        return int(end - self.started)

    def choose_size(self):

```

```

while True:
    raw = input(f"Board size {SIZES}: ").strip()
    if raw.isdigit() and int(raw) in SIZES:
        return int(raw)
    print("Please enter 3, 4 or 5.")

def display(self):
    width = len(str(self.size * self.size - 1))
    print()
    for row in self.puzzle.board:
        print(" ".join(f"{x or '':>{width}}" for x in row))
    print("Moves:", self.moves, " Time:", self.elapsed(), "s")

def run(self):
    print("Sliding Puzzle – W/A/S/D moves the tile into the blank. Q quits.")
    self.start_session(self.choose_size())
    while True:
        self.display()
        if self.puzzle.solved():
            print(f"Solved in {self.moves} moves!")
            return
        key = input("> ").strip().lower()
        if key == "q":
            return
        if key not in ("w", "a", "s", "d"):
            print("Use W/A/S/D (or Q to quit).")
            continue
        tile = self.puzzle.move(key)
        if tile is None:
            print("That move is not possible.")
        else:
            self.moves += 1
            print(f"Slid tile {tile}.")
            if self.puzzle.solved():
                self.finished_at = time.monotonic()

```

main.py and requirements.txt stay unchanged.

Quick verification

1. Run `python check_parity.py` and confirm all zeros.
2. Run `python main.py`, pick a size, push into a wall (count stays 0, no "Slid tile"), then make a valid move (count goes to 1, "Slid tile N" appears).
3. Press Enter with nothing typed. It should say "Use W/A/S/D" and not crash.

If check_parity.py still misbehaves, send me the exact error text and I'll pin it down.

```

Traceback (most recent call last):
  File "c:\Users\Adithya
Ajeeth\Downloads\22_sliding_puzzle\check_parity.py", line 17, in
<module>
    p = Puzzle(n)
  File "c:\Users\Adithya
Ajeeth\Downloads\22_sliding_puzzle\puzzle.py", line 23, in __init__
    self.board = self.make_board()
    ~~~~~^
  File "c:\Users\Adithya
Ajeeth\Downloads\22_sliding_puzzle\puzzle.py", line 33, in
make_board
    self._slide(last)
    ^^^^^^^^^
AttributeError: 'Puzzle' object has no attribute '_slide'

im getting this error when i run check_parity.py

```



The error means `puzzle.py` has no `_slide` method. In the task-by-task walkthrough I showed `_slide` and `move` as a snippet, and it's easy to miss that they replace the old `move()`. Your file most likely still has the original `move()` and nothing named `_slide`.

The fix

In `puzzle.py`, **delete the old `move()` method** and put these two in its place, inside the `Puzzle` class:

python

```

def _slide(self, direction):
    if not self._can_slide(direction):
        return None
    r, c, nr, nc = self._target(direction)
    tile = self.board[nr][nc]
    self.board[r][c], self.board[nr][nc] = tile, 0
    return tile

def move(self, direction):

```

```

"""Return the tile that slid into the blank, or None if nothing moved."""
if self.solved() or direction not in DIRECTIONS:
    return None
return self._slide(direction)

```

Safest option: replace the whole file

Since pieces have been going missing, paste this complete `puzzle.py` over your current one. It contains every method and constant in the right place:

python

```

import random

SIZES = (3, 4, 5)
DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}

class Puzzle:
    def __init__(self, size=4):
        if size not in SIZES:
            raise ValueError(f"size must be one of {SIZES}")
        self.size = size
        self.board = self.make_board()

    def solved_board(self):
        n = self.size
        tiles = list(range(1, n * n)) + [0]
        return [tiles[r * n:(r + 1) * n] for r in range(n)]

    def make_board(self):
        # Scramble with legal moves only, so the result is always solvable.
        while True:
            self.board = self.solved_board()
            last = None
            for _ in range(self.size * self.size * 20):
                options = [d for d in DIRECTIONS
                           if d != OPPOSITE.get(last) and self._can_slide(d)]
                last = random.choice(options)
                self._slide(last)
            if not self.solved():
                return self.board

    def blank_pos(self):
        for r in range(self.size):
            for c in range(self.size):
                if self.board[r][c] == 0:
                    return r, c

```

```

def _target(self, direction):
    r, c = self.blank_pos()
    dr, dc = DIRECTIONS[direction]
    return r, c, r + dr, c + dc

def _can_slide(self, direction):
    _, _, nr, nc = self._target(direction)
    return 0 <= nr < self.size and 0 <= nc < self.size

def _slide(self, direction):
    if not self._can_slide(direction):
        return None
    r, c, nr, nc = self._target(direction)
    tile = self.board[nr][nc]
    self.board[r][c], self.board[nr][nc] = tile, 0
    return tile

def move(self, direction):
    """Return the tile that slid into the blank, or None if nothing moved."""
    if self.solved() or direction not in DIRECTIONS:
        return None
    return self._slide(direction)

def solved(self):
    return self.board == self.solved_board()

```

Check it

```
bash
```

```
python check_parity.py
```

You should see three lines, one per size, each showing `unsolvable=0/1000, started solved=0/1000` . Then run `python main.py` to try the game itself.

One more thing: your `.pyc` files (`game_cpython-313.pyc` and `puzzle_cpython-313.pyc`) are compiled copies of the old code. Python regenerates its own in `__pycache__` , so they shouldn't interfere, but you can safely ignore or delete them.

If another error appears, paste the traceback and I'll fix it.





`puzzle.py` is now finished. It already covers the puzzle-side parts of Tasks 2 and 4 (the solved-state guard and `move()` returning the tile or `None`). What remains is `game.py`, which holds the rest of Tasks 2, 3 and 4.

Step 1: replace `game.py`

Paste this over the whole file:

python

```
import time
from puzzle import Puzzle, SIZES

class SlidingPuzzle:
    def __init__(self):
        self.size = 4
        self.puzzle = None
        self.moves = 0
        self.started = None
        self.finished_at = None

    def new_board(self):          # board only; session values untouched
        self.puzzle = Puzzle(self.size)

    def start_session(self, size): # the only place counters are reset
        self.size = size
        self.new_board()
        self.moves = 0
        self.started = time.monotonic()
        self.finished_at = None

    def elapsed(self):
        end = self.finished_at if self.finished_at is not None else time.monotonic()
        return int(end - self.started)

    def choose_size(self):
        while True:
            raw = input(f"Board size {SIZES}: ").strip()
            if raw.isdigit() and int(raw) in SIZES:
                return int(raw)
            print("Please enter 3, 4 or 5.")
```

```

def display(self):
    width = len(str(self.size * self.size - 1))
    print()
    for row in self.puzzle.board:
        print(" ".join(f"{x or '':>{width}}" for x in row))
    print("Moves:", self.moves, " Time:", self.elapsed(), "s")

def run(self):
    print("Sliding Puzzle - W/A/S/D moves the tile into the blank. Q quits.")
    self.start_session(self.choose_size())
    while True:
        self.display()
        if self.puzzle.solved():
            print(f"Solved in {self.moves} moves!")
            return
        key = input("> ").strip().lower()
        if key == "q":
            return
        if key not in ("w", "a", "s", "d"):
            print("Use W/A/S/D (or Q to quit).")
            continue
        tile = self.puzzle.move(key)
        if tile is None:
            print("That move is not possible.")
        else:
            self.moves += 1
            print(f"Slid tile {tile}.")
            if self.puzzle.solved():
                self.finished_at = time.monotonic()

```

What each task maps to

Task 4 (valid-action feedback)

- `tile = self.puzzle.move(key)` returns the tile that slid, or `None` .
- The counter and the "Slid tile" message sit together in the `else` branch, so an invalid move can never increment the count or print success.
- `key not in ("w", "a", "s", "d")` is a tuple check, so Enter or `wa` no longer crashes.

Task 2 (lifecycle)

- After a move that solves the board, `finished_at` is recorded. `elapsed()` then returns that frozen value, so the timer stops.
- The loop prints "Solved in N moves!" and returns, which ends the game cleanly.
- Commands can't change a finished board, because `Puzzle.move()` returns `None` when `solved()` is true.

Task 3 (sizes and session state)

- `choose_size()` accepts only 3, 4 or 5 and re-asks otherwise.
- `new_board()` only replaces the puzzle. Only `start_session()` resets moves and time, so recreating a board can't wipe them.
- Tile width is computed from the size, so 5x5 aligns.

Step 2: test it

Save this as `test_game.py` in the same folder and run `python test_game.py`. It covers the README's testing list without needing to play by hand:

python

```
import time
import unittest
from game import SlidingPuzzle
from puzzle import Puzzle

class Tests(unittest.TestCase):
    def test_all_sizes_start_unsolved(self):
        for n in (3, 4, 5):
            for _ in range(200):
                self.assertFalse(Puzzle(n).solved())

    def test_bad_size_rejected(self):
        with self.assertRaises(ValueError):
            Puzzle(6)

    def test_invalid_move_returns_none(self):
        p = Puzzle(3)
        p.board = [[1, 2, 3], [4, 5, 6], [7, 8, 0]] # blank bottom-right
        p.board = [[1, 2, 3], [4, 5, 6], [8, 7, 0]] # not solved, so moves are allowed
        self.assertIsNone(p.move("s")) # wall below
        self.assertIsNone(p.move("d")) # wall right
        self.assertIsNone(p.move("x")) # bad key

    def test_valid_move_returns_tile(self):
        p = Puzzle(3)
        p.board = [[1, 2, 3], [4, 5, 6], [8, 7, 0]]
        self.assertEqual(p.move("a"), 7)

    def test_solving_locks_board(self):
        p = Puzzle(3)
        p.board = [[1, 2, 3], [4, 5, 6], [7, 0, 8]]
        self.assertEqual(p.move("d"), 8) # this move solves it
        self.assertTrue(p.solved())
```

```

self.assertIsNone(p.move("a"))                # locked after completion
self.assertEqual(p.board, p.solved_board())

def test_new_board_keeps_session_values(self):
    g = SlidingPuzzle()
    g.start_session(3)
    g.moves = 5
    started = g.started
    g.new_board()
    self.assertEqual(g.moves, 5)
    self.assertEqual(g.started, started)

def test_timer_freezes(self):
    g = SlidingPuzzle()
    g.start_session(3)
    g.finished_at = g.started + 7
    self.assertEqual(g.elapsed(), 7)
    time.sleep(1.1)
    self.assertEqual(g.elapsed(), 7)

if __name__ == "__main__":
    unittest.main()

```

Expected result: Ran 7 tests ... OK .

Step 3: manual checks for the "after" video

Run `python main.py` and do these in order:

1. Enter `3` . The 3x3 board should appear.
2. Press Enter with nothing typed. You should see "Use W/A/S/D" and no crash.
3. Push into a wall. You should see "That move is not possible." and Moves stays 0.
4. Make a valid move. You should see "Slid tile N." and Moves becomes 1.
5. Watch the timer tick up between prompts.
6. To see a win quickly, temporarily change `self.size * self.size * 20` in `puzzle.py` to `2` , solve the tiny scramble, and confirm "Solved in N moves!" with a frozen time. Change it back to `20` afterwards.
7. Rerun and try sizes 4 and 5 to check alignment, then `q` to quit.

Submission reminder

- **Before video:** use the original code (keep a copy of the untouched files) and show the unsolvable boards or the Enter-key crash.
- **After video:** show steps 1 to 4 above plus the solved screen, within 10 seconds.

- Include the full chat link.

Run `test_game.py` and tell me what it prints. If anything fails, paste the output and I'll pin it down.

2026-09-30 08:55:27 You 

I am supposed to do each task one by one and commit the changes. Show me the changes i need to make for one task and do not give me the instructions for the next task unless i tell you to



Task 1: Guarantee solvable starting states

This version changes only what Task 1 needs, so your commit stays clean. `game.py` doesn't change at all for this task.

Before you edit

If you haven't recorded the "before" video, do it now with the original code. Run `python main.py` a few times and show an unsolvable board.

The change: `puzzle.py`

Replace the whole file with this:

```
python

import random

DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}

class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def solved_board(self):
        n = self.size
        tiles = list(range(1, n * n)) + [0]
        return [tiles[r * n:(r + 1) * n] for r in range(n)]

    def make_board(self):
```

```

# Scramble the solved board with legal blank moves only,
# so every generated board is reachable (solvable) by construction.
while True:
    self.board = self.solved_board()
    last = None
    for _ in range(self.size * self.size * 20):
        options = [d for d in DIRECTIONS
                    if d != OPPOSITE.get(last) and self._can_slide(d)]
        last = random.choice(options)
        self._slide(last)
    if not self.solved():
        return self.board

def blank_pos(self):
    for r in range(self.size):
        for c in range(self.size):
            if self.board[r][c] == 0:
                return r, c

def _target(self, direction):
    r, c = self.blank_pos()
    dr, dc = DIRECTIONS[direction]
    return r, c, r + dr, c + dc

def _can_slide(self, direction):
    _, _, nr, nc = self._target(direction)
    return 0 <= nr < self.size and 0 <= nc < self.size

def _slide(self, direction):
    if not self._can_slide(direction):
        return None
    r, c, nr, nc = self._target(direction)
    tile = self.board[nr][nc]
    self.board[r][c], self.board[nr][nc] = tile, 0
    return tile

def move(self, direction):
    return self._slide(direction) is not None

def solved(self):
    return self.board == self.solved_board()

```

What changed and why

- **make_board** no longer calls `random.shuffle`. It builds the solved board and applies many random legal moves. Every legal move preserves solvability, so every result is solvable.
- **OPPOSITE** stops the scramble from undoing its previous move, which would waste steps.

- The `while True` loop re-scrambles in the rare case the result ends up solved, so a game never starts already finished.
- `_target` , `_can_slide` , `_slide` are small helpers the scramble needs. `move()` still returns `True` or `False` like the original, so `game.py` keeps working untouched.

Test it

Save as `check_parity.py` in the same folder and run `python check_parity.py` :

python

```
from puzzle import Puzzle

def solvable(board, n):
    flat = [x for row in board for x in row]
    tiles = [x for x in flat if x]
    inv = sum(1 for i in range(len(tiles))
              for j in range(i + 1, len(tiles)) if tiles[i] > tiles[j])
    if n % 2 == 1:
        return inv % 2 == 0
    blank_row_from_bottom = n - flat.index(0) // n
    return (inv + blank_row_from_bottom) % 2 == 1

for n in (3, 4, 5):
    bad = started_solved = 0
    for _ in range(1000):
        p = Puzzle(n)
        bad += not solvable(p.board, n)
        started_solved += p.solved()
    print(f"{n}x{n}: unsolvable={bad}/1000, started solved={started_solved}/1000")
```

Expected output is `0/1000` and `0/1000` on all three lines. With the original `random.shuffle` code, the unsolvable count is around 500 per line. That is your reproduced defect.

Then run `python main.py` and confirm the game still starts and moves work.

Commit

bash

```
git add puzzle.py check_parity.py
git commit -m "Task 1: generate solvable boards by scrambling with legal moves"
```

Tell me when Task 1 is committed and I'll give you Task 2.

[do for task 2 now](#)

Task 2: Complete puzzle lifecycle

Your current code already detects the solved board and exits the loop. Two gaps remain:

1. **Puzzle.move() doesn't refuse moves after completion.** Nothing at the puzzle level stops a solved board from being changed.
2. **The timer keeps running after the win.** `display()` always calls `time.monotonic()`, so the final time is wrong.

Only `puzzle.py` and `game.py` change.

Change 1: `puzzle.py`

Replace the `move` method with:

```
python

def move(self, direction):
    if self.solved():          # locked once the puzzle is complete
        return False
    return self._slide(direction) is not None
```

The guard lives in `Puzzle`, so no caller can alter a finished board.

Change 2: `game.py`

Replace the whole file with this. It is your original structure plus the finish-time tracking:

```
python

import time
from puzzle import Puzzle

class SlidingPuzzle:
    def __init__(self):
        self.size = 4
        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()
        self.finished_at = None
```

```

def elapsed(self):
    end = self.finished_at if self.finished_at is not None else time.monotonic()
    return int(end - self.started)

def display(self):
    print()
    for row in self.puzzle.board:
        print(" ".join(f"{x or ' ':>2}" for x in row))
    print("Moves:", self.moves, " Time:", self.elapsed(), "s")

def run(self):
    print("Sliding Puzzle – W/A/S/D moves the tile into the blank. Q quits.")
    while True:
        self.display()
        if self.puzzle.solved():
            print(f"Solved in {self.moves} moves and {self.elapsed()}s!")
            return
        key = input("> ").strip().lower()
        if key == "q":
            return
        if key not in "wasd":
            print("Use W/A/S/D.")
            continue
        if self.puzzle.move(key):
            self.moves += 1
            if self.puzzle.solved():
                self.finished_at = time.monotonic() # freeze the timer
        else:
            print("That move is not possible.")

```

What changed and why

- **finished_at** and **elapsed()** store the moment of solving, then return that frozen value, so the time stops counting.
- The **if self.puzzle.solved()** check after a successful move records the finish time exactly once, right when the winning move lands.
- The end message now shows moves and time, and the loop returns, so no further input is read.
- The guard in **Puzzle.move()** is a second line of defence: even if some future code called **move()** after the win, the board would not change.

Test it

Automated: save as `test_task2.py` and run `python test_task2.py` :

```
python
```

```

import time
import unittest
from game import SlidingPuzzle
from puzzle import Puzzle

class Task2Tests(unittest.TestCase):
    def make_near_solved(self):
        p = Puzzle(3)
        p.board = [[1, 2, 3], [4, 5, 6], [7, 0, 8]] # one move from solved
        return p

    def test_winning_move_solves(self):
        p = self.make_near_solved()
        self.assertTrue(p.move("d"))
        self.assertTrue(p.solved())

    def test_board_locked_after_solve(self):
        p = self.make_near_solved()
        p.move("d")
        before = [row[:] for row in p.board]
        for key in "wasd":
            self.assertFalse(p.move(key))
        self.assertEqual(p.board, before)

    def test_timer_freezes(self):
        g = SlidingPuzzle()
        g.finished_at = g.started + 7
        self.assertEqual(g.elapsed(), 7)
        time.sleep(1.1)
        self.assertEqual(g.elapsed(), 7)

if __name__ == "__main__":
    unittest.main()

```

Expected: Ran 3 tests ... OK .

Manual: to reach a win quickly, temporarily change `self.size * self.size * 20` in `make_board` to `2` and set `self.size = 3` in `game.py` . Solve the tiny scramble and confirm you see "Solved in N moves and Ns!" and the game exits. Then **revert both temporary edits** before committing.

Commit

bash

```

git add puzzle.py game.py test_task2.py
git commit -m "Task 2: lock board after solve, freeze timer, end game cleanly"

```

Tell me when Task 2 is committed and I'll give you Task 3.

2026-09-30 11:48:56

You 

give for task 3 now



Task 3: Size, timer, and move modes

The goal is 3x3, 4x4 and 5x5 modes, with moves and time tracked the same way in all of them. Recreating a board must not reset the session values.

The design splits the state in two:

- **Board state** is the tile layout.
- **Session state** is `moves`, `started` and `finished_at`.

Only `start_session()` resets the session values. `new_board()` touches only the board.

Change 1: `puzzle.py`

Add `SIZES` under the other constants at the top:

```
python
```

```
SIZES = (3, 4, 5)
```

Then replace `__init__` with a version that rejects unsupported sizes:

```
python
```

```
def __init__(self, size=4):
    if size not in SIZES:
        raise ValueError(f"size must be one of {SIZES}")
    self.size = size
    self.board = self.make_board()
```

Change 2: `game.py`

Replace the whole file with this:

```
python
```

```

import time
from puzzle import Puzzle, SIZES

class SlidingPuzzle:
    def __init__(self):
        self.size = 4
        self.puzzle = None
        self.moves = 0
        self.started = None
        self.finished_at = None

    def new_board(self):          # board only; session values untouched
        self.puzzle = Puzzle(self.size)

    def start_session(self, size): # the only place counters are reset
        self.size = size
        self.new_board()
        self.moves = 0
        self.started = time.monotonic()
        self.finished_at = None

    def elapsed(self):
        end = self.finished_at if self.finished_at is not None else time.monotonic()
        return int(end - self.started)

    def choose_size(self):
        while True:
            raw = input(f"Board size {SIZES}: ").strip()
            if raw.isdigit() and int(raw) in SIZES:
                return int(raw)
            print("Please enter 3, 4 or 5.")

    def display(self):
        width = len(str(self.size * self.size - 1))
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or '':>{width}}" for x in row))
        print("Moves:", self.moves, " Time:", self.elapsed(), "s")

    def run(self):
        print("Sliding Puzzle - W/A/S/D moves the tile into the blank. Q quits.")
        self.start_session(self.choose_size())
        while True:
            self.display()
            if self.puzzle.solved():
                print(f"Solved in {self.moves} moves and {self.elapsed()}s!")
                return

```

```

key = input("> ").strip().lower()
if key == "q":
    return
if key not in "wasd":
    print("Use W/A/S/D.")
    continue
if self.puzzle.move(key):
    self.moves += 1
    if self.puzzle.solved():
        self.finished_at = time.monotonic() # freeze the timer
else:
    print("That move is not possible.")

```

What changed and why

- **new_board() vs start_session()** : this is the README's "do not reset session values accidentally" requirement. Recreating a board can no longer wipe the counters, because only `start_session()` does that.
- **choose_size()** : it uses `isdigit()` and a membership check, so `abc` , `7` , an empty line and `-3` are all rejected and re-asked.
- **Puzzle.__init__ validation**: an unsupported size can't get in through any route other than the menu.
- **Computed width** : the tile width comes from the largest tile number, so 3x3 (max 8), 4x4 (max 15) and 5x5 (max 24) all align.
- **One timer and one counter for every size**: they live in the session, not in anything size-specific, so behaviour is identical across modes.

Everything else, including the Task 2 timer freeze, is unchanged.

Test it

Automated: save as `test_task3.py` and run `python test_task3.py` :

python

```

import unittest
from game import SlidingPuzzle
from puzzle import Puzzle

class Task3Tests(unittest.TestCase):
    def test_all_sizes_build_correct_boards(self):
        for n in (3, 4, 5):
            p = Puzzle(n)
            self.assertEqual(len(p.board), n)
            self.assertTrue(all(len(row) == n for row in p.board))

```

```

        flat = sorted(x for row in p.board for x in row)
        self.assertEqual(flat, list(range(n * n)))

def test_unsupported_sizes_rejected(self):
    for bad in (2, 6, 0, -1):
        with self.assertRaises(ValueError):
            Puzzle(bad)

def test_new_board_keeps_session_values(self):
    g = SlidingPuzzle()
    g.start_session(3)
    g.moves = 5
    started = g.started
    g.new_board()
    self.assertEqual(g.moves, 5)
    self.assertEqual(g.started, started)

def test_start_session_resets_and_sets_size(self):
    g = SlidingPuzzle()
    g.start_session(3)
    g.moves = 9
    g.finished_at = g.started + 1
    g.start_session(5)
    self.assertEqual(g.size, 5)
    self.assertEqual(len(g.puzzle.board), 5)
    self.assertEqual(g.moves, 0)
    self.assertIsNone(g.finished_at)

def test_timer_runs_for_every_size(self):
    for n in (3, 4, 5):
        g = SlidingPuzzle()
        g.start_session(n)
        self.assertGreaterEqual(g.elapsed(), 0)

if __name__ == "__main__":
    unittest.main()

```

Expected: Ran 5 tests ... OK .

Manual: run `python main.py` and check these:

1. Type `abc` , then `7` , then press Enter, and finally `3` . The first three should be rejected, and `3` should start a 3x3 game.
2. Check the alignment and that Time increases between prompts.
3. Make a valid move and check that Moves goes to 1.
4. Quit with `q` , rerun and try `4` and `5` . Check that the 5x5 columns line up with two-digit tiles.

Commit

bash

```
git add puzzle.py game.py test_task3.py
git commit -m "Task 3: add 3x3/4x4/5x5 modes, separate board and session state"
```

Tell me when Task 3 is committed and I'll give you Task 4.

2026-09-30 11:52:43

You 

now give for task 4



Task 4: Valid-action feedback

Right now `move()` returns `True` or `False`, and the game never tells the player what happened on a successful move. There is also a latent bug: `key not in "wasd"` is a substring test, so pressing Enter (`"`) or typing `wa` passes the check and then crashes `move()` with a `KeyError`.

The fix is to make `move()` return the tile that slid (or `None` if nothing moved) and have the game report success only when it gets a tile back.

Change 1: `puzzle.py`

Replace the `move` method with:

python

```
def move(self, direction):
    """Return the tile that slid into the blank, or None if nothing moved."""
    if self.solved() or direction not in DIRECTIONS:
        return None
    return self._slide(direction)
```

- `direction not in DIRECTIONS` means an unknown key can never reach `_target`, so the `KeyError` is impossible at the puzzle level.
- Tiles are numbered from 1, so a returned tile is never `0`, and `None` unambiguously means "no move".

Change 2: `game.py`

Only the input check and the move handling in `run()` change. Replace these lines:

python

```
if key not in "wasd":
    print("Use W/A/S/D.")
    continue
if self.puzzle.move(key):
    self.moves += 1
    if self.puzzle.solved():
        self.finished_at = time.monotonic() # freeze the timer
else:
    print("That move is not possible.")
```

with:

python

```
if key not in ("w", "a", "s", "d"):
    print("Use W/A/S/D (or Q to quit).")
    continue
tile = self.puzzle.move(key)
if tile is None:
    print("That move is not possible.")
else:
    self.moves += 1
    print(f"Slid tile {tile}.")
    if self.puzzle.solved():
        self.finished_at = time.monotonic() # freeze the timer
```

What changed and why

- **Tuple membership** (`in ("w", "a", "s", "d")`) requires an exact match, so `""`, `wa` and `12` are rejected with a message instead of crashing.
- `tile = self.puzzle.move(key)` gives the game real evidence of what happened.
- **The counter and the success message share the `else` branch**, so they can only ever occur together, and an invalid move can never increment `moves` or print "Slid tile".

Test it

Automated: save as `test_task4.py` and run `python test_task4.py` :

python

```
import unittest
from puzzle import Puzzle
```

```

class Task4Tests(unittest.TestCase):
    def setUp(self):
        self.p = Puzzle(3)
        self.p.board = [[1, 2, 3], [4, 5, 6], [8, 7, 0]] # blank bottom-right, not solved

    def test_valid_move_returns_tile(self):
        self.assertEqual(self.p.move("a"), 7) # 7 slides right into the blank
        self.assertEqual(self.p.board[2], [8, 0, 7])

    def test_wall_moves_return_none_and_change_nothing(self):
        before = [row[:] for row in self.p.board]
        self.assertIsNone(self.p.move("s")) # wall below
        self.assertIsNone(self.p.move("d")) # wall right
        self.assertEqual(self.p.board, before)

    def test_invalid_keys_return_none(self):
        before = [row[:] for row in self.p.board]
        for bad in ("", "x", "wa", "12"):
            self.assertIsNone(self.p.move(bad))
        self.assertEqual(self.p.board, before)

if __name__ == "__main__":
    unittest.main()

```

Expected: Ran 3 tests ... OK . Your earlier `test_task2.py` should still pass too, since a returned tile is truthy and `None` is falsy. Run both to confirm.

Manual: run `python main.py` , pick a size, then:

1. Push into a wall several times. You should see "That move is not possible." and Moves stays at 0.
2. Make a valid move. You should see "Slid tile N." and Moves becomes 1.
3. Press Enter with nothing typed, then type `wa` , `x` and `12` . Each should show "Use W/A/S/D (or Q to quit)." with no crash and no change to Moves.
4. Make 3 valid and 2 invalid moves in any order. Moves should read 3.

Commit

bash

```

git add puzzle.py game.py test_task4.py
git commit -m "Task 4: report success only when a tile moves; fix invalid-input crash"

```

That completes all four tasks. When you're ready, I can walk you through the final checks: the "after" video plan, and a quick review of the whole project against the README's submission checklist.

```
py -u "c:\Users\Adithya Ajeeth\Downloads\SE-Lab04\test_task4.py"
EEE
```

```
=====
=====
```

```
ERROR: test_invalid_keys_return_none
(__main__.Task4Tests.test_invalid_keys_return_none)
```

Traceback (most recent call last):

File "c:\Users\Adithya Ajeeth\Downloads\SE-Lab04\test_task4.py",
line 7, in setUp

```
    self.p = Puzzle(3)
              ~~~~~^^^
```

TypeError: Puzzle() takes no arguments

```
=====
=====
```

```
ERROR: test_valid_move_returns_tile
(__main__.Task4Tests.test_valid_move_returns_tile)
```

Traceback (most recent call last):

File "c:\Users\Adithya Ajeeth\Downloads\SE-Lab04\test_task4.py",
line 7, in setUp

```
    self.p = Puzzle(3)
              ~~~~~^^^
```

TypeError: Puzzle() takes no arguments

```
=====
=====
```

```
ERROR: test_wall_moves_return_none_and_change_nothing
(__main__.Task4Tests.test_wall_moves_return_none_and_change_n
othing)
```

Traceback (most recent call last):

File "c:\Users\Adithya Ajeeth\Downloads\SE-Lab04\test_task4.py",
line 7, in setUp

```
    self.p = Puzzle(3)
              ~~~~~^^^
```

TypeError: Puzzle() takes no arguments

Ran 3 tests in 0.004s

FAILED (errors=3)



This error isn't caused by the Task 4 test. `Puzzle()` takes no arguments means Python found a `Puzzle` class with **no `__init__` method**. So the `puzzle.py` in `SE-Lab04` doesn't have the constructor.

This is a different folder from the earlier `22_sliding_puzzle`, so it may be a copy that's missing an edit, or the `__init__` was lost when you replaced it in Task 3 (for example, deleted, or mis-indented so it sits outside the class).

Check it

From the `SE-Lab04` folder, run:

```
bash
```

```
python -c "import puzzle; print(puzzle.__file__); print(hasattr(puzzle.Puzzle, '__init__'), puzzle.Puzzle.__init__)"
```

Then open `puzzle.py` and look at the top of the class. It must look like this:

```
python
```

```
import random

SIZES = (3, 4, 5)
DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}

class Puzzle:
    def __init__(self, size=4):
        if size not in SIZES:
            raise ValueError(f"size must be one of {SIZES}")
        self.size = size
        self.board = self.make_board()
```

What to look for

- `__init__` is missing. Add it as shown above.
- `__init__` is spelled wrong. It needs two underscores on each side, so `_init_` or `__int__` won't work.
- Indentation is wrong. `def __init__` must be indented 4 spaces under `class Puzzle:`. If it sits at the left margin, it isn't part of the class.
- `SIZES` is missing. Task 3 adds it. Without it you'd get a `NameError` next.

Safest fix

If the file has drifted, replace `puzzle.py` with this complete version, which includes the Task 1, 2, 3 and 4 changes:

python

```
import random

SIZES = (3, 4, 5)
DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}

class Puzzle:
    def __init__(self, size=4):
        if size not in SIZES:
            raise ValueError(f"size must be one of {SIZES}")
        self.size = size
        self.board = self.make_board()

    def solved_board(self):
        n = self.size
        tiles = list(range(1, n * n)) + [0]
        return [tiles[r * n:(r + 1) * n] for r in range(n)]

    def make_board(self):
        # Scramble the solved board with legal blank moves only,
        # so every generated board is reachable (solvable) by construction.
        while True:
            self.board = self.solved_board()
            last = None
            for _ in range(self.size * self.size * 20):
                options = [d for d in DIRECTIONS
                           if d != OPPOSITE.get(last) and self._can_slide(d)]
                last = random.choice(options)
                self._slide(last)
            if not self.solved():
                return self.board
```

```

def blank_pos(self):
    for r in range(self.size):
        for c in range(self.size):
            if self.board[r][c] == 0:
                return r, c

def _target(self, direction):
    r, c = self.blank_pos()
    dr, dc = DIRECTIONS[direction]
    return r, c, r + dr, c + dc

def _can_slide(self, direction):
    _, _, nr, nc = self._target(direction)
    return 0 <= nr < self.size and 0 <= nc < self.size

def _slide(self, direction):
    if not self._can_slide(direction):
        return None
    r, c, nr, nc = self._target(direction)
    tile = self.board[nr][nc]
    self.board[r][c], self.board[nr][nc] = tile, 0
    return tile

def move(self, direction):
    """Return the tile that slid into the blank, or None if nothing moved."""
    if self.solved() or direction not in DIRECTIONS:
        return None
    return self._slide(direction)

def solved(self):
    return self.board == self.solved_board()

```

Then rerun:

```
bash
```

```
python test_task4.py
```

You should get `Ran 3 tests ... OK` . Also make sure `test_task4.py` and `puzzle.py` are in the same folder, and that no stray `puzzle.py` or `puzzle` folder sits elsewhere on your path. The first command above prints which file Python actually imported.

If it still fails, paste the output of that first command and the top 10 lines of your `puzzle.py` .

```
py -u "c:\Users\Adithya Ajeeth\Downloads\SE-Lab04\test_task4.py"
```

```
EFF
```

```
=====
```

```
=====
```

```
ERROR: test_invalid_keys_return_none
```

```
(__main__.Task4Tests.test_invalid_keys_return_none)
```

```
Traceback (most recent call last):
```

```
File "c:\Users\Adithya Ajeeth\Downloads\SE-
```

```
Lab04\test_task4.py",line 23, in test_invalid_keys_return_none
```

```
self.assertIsNone(self.p.move(bad))
```

```
~~~~~^
```

```
File "c:\Users\Adithya Ajeeth\Downloads\SE-Lab04\puzzle.py",
```

```
line 65, in move
```

```
return self._slide(direction) is not None
```

```
~~~~~^
```

```
File "c:\Users\Adithya Ajeeth\Downloads\SE-Lab04\puzzle.py",
```

```
line 55, in _slide
```

```
if not self._can_slide(direction):
```

```
~~~~~^
```

```
File "c:\Users\Adithya Ajeeth\Downloads\SE-Lab04\puzzle.py",
```

```
line 51, in _can_slide
```

```
_, _, nr, nc = self._target(direction)
```

```
~~~~~^
```

```
File "c:\Users\Adithya Ajeeth\Downloads\SE-Lab04\puzzle.py",
```

```
line 47, in _target
```

```
dr, dc = DIRECTIONS[direction]
```

```
~~~~~^
```

```
KeyError: "
```

```
=====
```

```
=====
```

```
FAIL: test_valid_move_returns_tile
```

```
(__main__.Task4Tests.test_valid_move_returns_tile)
```

```
Traceback (most recent call last):
```

```
File "c:\Users\Adithya Ajeeth\Downloads\SE-
```

```
Lab04\test_task4.py",line 11, in test_valid_move_returns_tile
```

```
self.assertEqual(self.p.move("a"), 7) # 7 slides right into the  
blank
```

```
~~~~~^
```

```
AssertionError: True != 7
```

```
=====
=====
```

```
FAIL: test_wall_moves_return_none_and_change_nothing
(__main__.Task4Tests.test_wall_moves_return_none_and_change_n
othing)
```

```
Traceback (most recent call last):
```

```
File "c:\Users\Adithya Ajeeth\Downloads\SE-
Lab04\test_task4.py",line 16, in
```

```
test_wall_moves_return_none_and_change_nothing
```

```
    self.assertIsNone(self.p.move("s"))    # wall below
```

```
~~~~~^~~~~~
```

```
AssertionError: False is not None
```

```
Ran 3 tests in 0.005s
```

```
FAILED (failures=2, errors=1)
```



Good news: the constructor problem is fixed, and these three failures all point to the same thing. Your `puzzle.py` still has the **Task 2 version of `move()`**, so the Task 4 change was never applied.

The traceback shows it. Line 65 of your file reads:

```
python
```

```
return self._slide(direction) is not None
```

That's the old version, which returns `True` or `False`. The Task 4 version returns the tile or `None`. That explains each failure:

- `True != 7` : the old `move()` returns `True`, and the test expects the tile number `7`.
- `False is not None` : the old `move()` returns `False` for a wall move, and the test expects `None`.
- `KeyError: "` : the old `move()` has no `direction not in DIRECTIONS` check, so an empty or bad key reaches `DIRECTIONS[direction]` and crashes.

The fix

In `puzzle.py`, find the `move` method (around line 60) and replace it entirely with:

```
python
```

```
def move(self, direction):
    """Return the tile that slid into the blank, or None if nothing moved."""
    if self.solved() or direction not in DIRECTIONS:
        return None
    return self._slide(direction)
```

Nothing else in `puzzle.py` changes. Save the file and rerun:

```
bash
```

```
py test_task4.py
```

Expected: Ran 3 tests ... OK .

Also check `game.py`

Task 4 has a second half in `game.py`. Make sure the input check and move handling in `run()` look like this:

```
python
```

```
if key not in ("w", "a", "s", "d"):
    print("Use W/A/S/D (or Q to quit).")
    continue
tile = self.puzzle.move(key)
if tile is None:
    print("That move is not possible.")
else:
    self.moves += 1
    print(f"Slid tile {tile}.")
    if self.puzzle.solved():
        self.finished_at = time.monotonic() # freeze the timer
```

If `game.py` still has `if self.puzzle.move(key):`, it will still work with the new `move()`, but you won't get the "Slid tile N." feedback and the "wasd" substring bug will remain.

Once `test_task4.py` passes, rerun `test_task2.py` and `test_task3.py` as well. All three should pass with the new `move()`.

Then commit:

```
bash
```

```
git add puzzle.py game.py test_task4.py
git commit -m "Task 4: report success only when a tile moves; fix invalid-input crash"
```

If a test still fails after the edit, paste the new output.

2026-09-30 12:01:30 You 

```
[] puzzle.py
>[] game.py
>[] test_task4.py
im still getting the same error
```



I can see the cause in the `puzzle.py` you just uploaded: it has **two move methods**. The new one sits near the top, right after `__init__`, and the old Task 2 one sits lower down, just above `solved`. In Python, when a class defines the same method name twice, the **last definition wins**. So the old `move()` (returns `True / False`) silently overrides the new one, and that's why you keep seeing the same failures.

The fix

Delete this old method entirely, near the bottom of `puzzle.py`:

python

```
def move(self, direction):
    if self.solved():           # locked once the puzzle is complete
        return False
    return self._slide(direction) is not None
```

Keep the one near the top (the one with the docstring). Search the file for `def move`. You should end up with exactly **one** match.

Or paste the clean file

python

```
import random

SIZES = (3, 4, 5)
DIRECTIONS = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}
OPPOSITE = {"w": "s", "s": "w", "a": "d", "d": "a"}
```

```

class Puzzle:
    def __init__(self, size=4):
        if size not in SIZES:
            raise ValueError(f"size must be one of {SIZES}")
        self.size = size
        self.board = self.make_board()

    def solved_board(self):
        n = self.size
        tiles = list(range(1, n * n)) + [0]
        return [tiles[r * n:(r + 1) * n] for r in range(n)]

    def make_board(self):
        # Scramble with legal moves only, so the result is always solvable.
        while True:
            self.board = self.solved_board()
            last = None
            for _ in range(self.size * self.size * 20):
                options = [d for d in DIRECTIONS
                           if d != OPPOSITE.get(last) and self._can_slide(d)]
                last = random.choice(options)
                self._slide(last)
            if not self.solved():
                return self.board

    def blank_pos(self):
        for r in range(self.size):
            for c in range(self.size):
                if self.board[r][c] == 0:
                    return r, c

    def _target(self, direction):
        r, c = self.blank_pos()
        dr, dc = DIRECTIONS[direction]
        return r, c, r + dr, c + dc

    def _can_slide(self, direction):
        _, _, nr, nc = self._target(direction)
        return 0 <= nr < self.size and 0 <= nc < self.size

    def _slide(self, direction):
        if not self._can_slide(direction):
            return None
        r, c, nr, nc = self._target(direction)
        tile = self.board[nr][nc]
        self.board[r][c], self.board[nr][nc] = tile, 0
        return tile

    def move(self, direction):

```

```
"""Return the tile that slid into the blank, or None if nothing moved."""
if self.solved() or direction not in DIRECTIONS:
    return None
return self._slide(direction)

def solved(self):
    return self.board == self.solved_board()
```

Your `game.py` and `test_task4.py` are already correct, so leave them alone.

Verify and commit

```
bash
```

```
py test_task4.py
```

Expected: Ran 3 tests ... OK . Then run `py test_task2.py` and `py test_task3.py` as well. Task 2's `test_winning_move_solves` uses `assertTrue` , which a returned tile (truthy) still satisfies, and `test_board_locked_after_solve` uses `assertFalse` , which `None` satisfies. So all should pass.

```
bash
```

```
git add puzzle.py game.py test_task4.py
git commit -m "Task 4: report success only when a tile moves; fix invalid-input crash"
```

The likely cause is that the new `move` got pasted in without deleting the old one. That's my fault for saying "replace" without stressing that the old method has to go.